Prática - Exceções

Exemplo Contas

- Estamos forçando uma conta a ter valor negativo:

```
Conta minhaConta = new Conta();
minhaConta.deposita(100);
minhaConta.setLimite(100);
minhaConta.saca(1000);
//o saldo é -900? É 100? É 0? A chamada ao método saca funcionou?
```

Exemplo Contas

- A solução mais simples utilizada antigamente é a de marcar o retorno de um método como boolean e retornar true, se tudo ocorreu da maneira planejada, ou false, caso contrário:

```
boolean saca(double quantidade) {
  // posso sacar até saldo+limite
  if (quantidade > this.saldo + this.limite) {
      System.out.println("Não posso sacar fora do limite!");
      return false;
    } else {
      this.saldo = this.saldo - quantidade;
      return true;
   }
 }
```

Exemplo Contas

- Nova forma de chamar o método:

```
Conta minhaConta = new Conta();

minhaConta.deposita(100);
minhaConta.setLimite(100);

if (!minhaConta.saca(1000)) {
  System.out.println("Não saquei");
}
```

Exemplo Contas

- Tivemos que lembrar de testar o retorno do método.
- Esquecer de testar teria consequencias drásticas: uma máquina de autoatendimento poderia liberar o dinheiro sem ter o saldo suficiente!

```
Conta minhaConta = new Conta();
minhaConta.deposita(100);

// ...
double valor = 5000;
minhaConta.saca(valor); // vai retornar false, mas ninguém verifica!
caixaEletronico.emite(valor);
```

Exemplo Contas

- Mesmo invocando o método e tratando o retorno de maneira correta, o que faríamos se fosse necessário sinalizar quando o usuário passou um valor negativo como quantidade?
- Uma solução seria alterar o retorno de **boolean** para **int** e retornar o código do erro que ocorreu.
  - Isso é considerado uma **má prática** (conhecida também como uso de "magic numbers").

Exceções

- Por esses e outros motivos, utilizamos um código diferente em Java para tratar aquilo que chamamos de exceções:
  - os casos onde acontece algo que, normalmente, não iria acontecer.
  - O exemplo do argumento do saque inválido é uma exceção à regra.

Exceções

- Para se familiarizar com os conceitos teste o seguinte código:

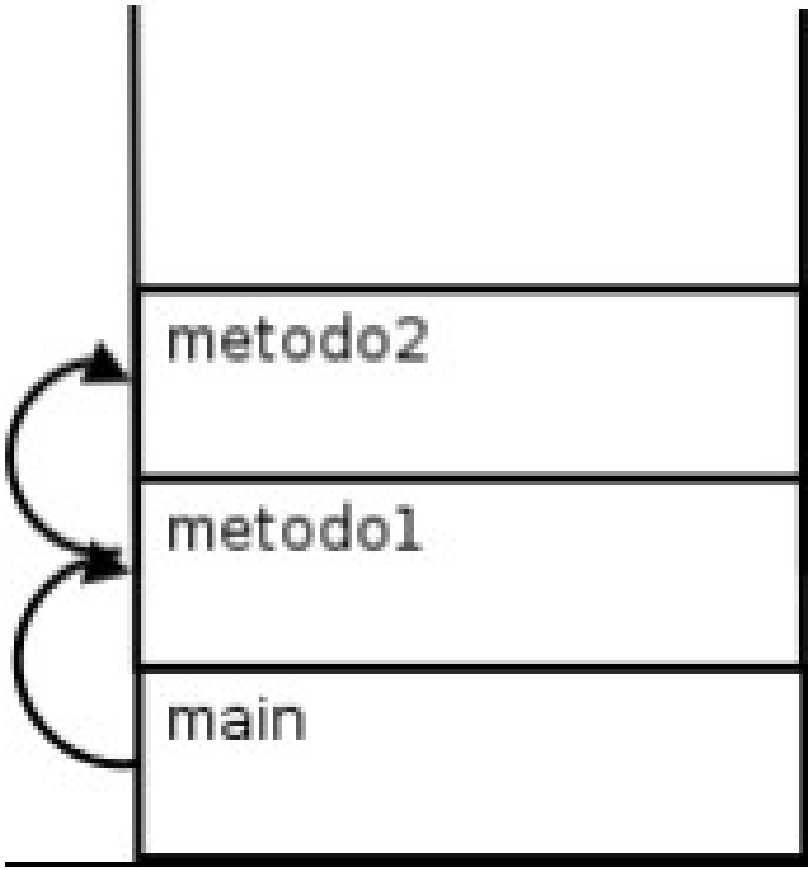
```java
class TesteErro {
  public static void main(String[] args) {
    System.out.println("inicio do main");
    metodo1();
    System.out.println("fim do main");
  }

  static void metodo1() {
    System.out.println("inicio do metodo1");
    metodo2();
    System.out.println("fim do metodo1");
  }

  static void metodo2() {
    System.out.println("inicio do metodo2");
    int[] array = new int[10];
    for (int i = 0; i <= 15; i++) {
      array[i] = i;
      System.out.println(i);
    }
    System.out.println("fim do metodo2");
  }
}
```

Exceções

- Toda invocação de método é empilhada... em uma estrutura de dados que isola a área de memória de cada um.
- Quando um método termina (retorna), ele volta para o método que o invocou. Ele descobre isso através da pilha de execução (stack). Basta jogar fora um gomo da pilha (stackframe):



Exceções

- O nosso metodo2 propositadamente possui um enorme problema: está acessando um índice de array indevido para esse caso; o índice estará fora dos limites da array quando chegar em 10!

- Rode o código. Qual é a saída? O que isso representa? O que ela indica?

Options

```
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
```

```
java.lang.ArrayIndexOutOfBoundsException: 10
        at TesteErro.metodo2(TesteErro.java:18)
        at TesteErro.metodo1(TesteErro.java:10)
        at TesteErro.main(TesteErro.java:4)
```

Exceções

- Esse é o conhecido rastro da pilha (**stacktrace**). É uma saída importantíssima para o programador - tanto que, em qualquer fórum ou lista de discussão, é comum os programadores enviarem, juntamente com a descrição do problema, essa stacktrace.

Exceções

- **Por que isso aconteceu?**
- O sistema de exceções do Java funciona da seguinte maneira:
  - quando uma exceção é lançada (throws), a JVM entra em estado de alerta e vai ver se o método atual toma alguma precaução ao tentar executar esse trecho de código. Como podemos ver, o metodo2 não toma nenhuma medida diferente do que vimos até agora.

Exceções

- Como o metodo2 não está tratando esse problema, a JVM pára a execução dele anormalmente, sem esperar ele terminar, e volta um stackframe pra baixo, onde será feita nova verificação:
  - o metodo1 está se precavendo de um problema chamado ArrayIndexOutOfBoundsException? Não... volta para o main, onde também não há proteção, então a JVM morre (na verdade, quem morre é apenas a Thread corrente).

Exceções

- Aqui estamos forçando esse caso, e não faria sentido tomarmos cuidado com ele. É fácil arrumar um problema desses: basta percorrermos a array no máximo até o seu length.
- Porém, apenas para entender o controle de fluxo de uma ***Exception***, vamos colocar o código que vai tentar (*try*) executar o bloco perigoso e, caso o problema seja do tipo *ArrayIndexOutOfBoundsException*, ele será pego (*caught*). Repare que é interessante que cada exceção no Java tenha um tipo... ela pode ter atributos e métodos.

Exceções

- Adicione um try/catch em volta do for, pegando ArrayIndexOutOfBoundsException.
  - O que o código imprime?

```
try {
  for (int i = 0; i <= 15; i++) {
    array[i] = i;
    System.out.println(i);
  }
} catch (ArrayIndexOutOfBoundsException e) {
  System.out.println("erro: " + e);
}
```

Options

```
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
erro: java.lang.ArrayIndexOutOfBoundsException: 10
fim do metodo2
fim do metodo1
fim do main
```

Exceções

- Em vez de fazer o try em torno do for inteiro, tente apenas com o bloco de dentro do for:

```
for (int i = 0; i <= 15; i++) {
  try {
    array[i] = i;
    System.out.println(i);
  } catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("erro: " + e);
  }
}
```

- Qual a diferença?

Options

```
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
erro: java.lang.ArrayIndexOutOfBoundsException: 10
erro: java.lang.ArrayIndexOutOfBoundsException: 11
erro: java.lang.ArrayIndexOutOfBoundsException: 12
erro: java.lang.ArrayIndexOutOfBoundsException: 13
erro: java.lang.ArrayIndexOutOfBoundsException: 14
erro: java.lang.ArrayIndexOutOfBoundsException: 15
fim do metodo2
fim do metodo1
fim do main
```

Exceções

- Retire o try/catch e coloque ele em volta da chamada do metodo2.

```
System.out.println("inicio do metodo1");
try {
  metodo2();
} catch (ArrayIndexOutOfBoundsException e) {
  System.out.println("erro: " + e);
}
System.out.println("fim do metodo1");
```

Options

```
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
erro: java.lang.ArrayIndexOutOfBoundsException: 10
fim do metodo1
fim do main
```

Options

```
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
erro: java.lang.ArrayIndexOutOfBoundsException: 10
fim do metodo1
fim do main
```

Cadê o fim do método 2???

Exceções

- Faça o mesmo só que agora na chamada do método 1 no main.
  - O que acontece?

```
System.out.println("inicio do main");
try {
  metodo1();
} catch (ArrayIndexOutOfBoundsException e) {
  System.out.println("Erro : "+e);
}
System.out.println("fim do main");
```

Options

```
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
Erro : java.lang.ArrayIndexOutOfBoundsException: 10
fim do main
```

Options

```
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
Erro : java.lang.ArrayIndexOutOfBoundsException: 10
fim do main
```

Cadê o fim dos métodos 2 e 1???

Exceções

- **Observação**:
  - A partir do momento que uma exceção foi pega (**catch**) a execução volta ao normal a partir daquele ponto.

Exceções Runtime

- Repare que um ArrayIndexOutOfBoundsException ou um NullPointerException poderia ser facilmente evitado com o **for** corretamente escrito ou com **ifs** que checariam os limites da array.
- Outro caso em que também ocorre tal tipo de exceção é quando um cast errado é feito.

Exceções Runtime

- Em todos os casos, tais problemas provavelmente poderiam ser evitados pelo programador. É por esse motivo que o java não te obriga a dar o try/catch nessas exceptions e chamamos essas exceções de unchecked (não verificadas).
- Em outras palavras, o compilador não checa se você está tratando essas exceções.

Exceções Verificadas

- Nos exemplos anteriores não fomos obrigados a tratar as exceções, código compilava normalmente (Exceções não verificadas/runtime/unchecked).
- Há outro tipo de exceção que obriga a quem chama o método ou construtor a tratá-la.
  - Chamamos esse tipo de exceção de verificada (checked), pois o compilador checará se ela está sendo devidamente tratada, diferente das anteriores, conhecidas como unchecked.

Exceções Verificadas

- Um exemplo interessante é o de abrir um arquivo para leitura, onde pode ocorrer o erro do arquivo não existir:

```
class Teste {
  public static void metodo() {
    new java.io.FileInputStream("arquivo.txt");
  }
}
```

- Tente compilar esse codigo.

Exceções Verificadas

Class Edit Tools Options

Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

```
class Teste {
  public static void metodo() {
    new java.io.FileInputStream("arquivo.txt");
  }
}
```

unreported exception java.io.FileNotFoundException; must be caught or declared to be thrown

?

saved

Exceções Verificadas

- Para compilar e fazer o programa funcionar, temos duas maneiras que podemos tratar o problema.
- O primeiro, é tratá-lo com o try e catch do mesmo jeito que usamos no exemplo anterior, com uma array:

```
try {
    new java.io.FileInputStream("arquivo.txt");
  } catch (java.io.FileNotFoundException e) {
    System.out.println("Nao foi possível abrir o arquivo para leitura");
  }
```

Exceções Verificadas

- A segunda forma de tratar esse erro, é delegar ele para quem chamou o nosso método, isto é, passar para a frente.

```
class Teste {

    public static void metodo() throws java.io.FileNotFoundException {
      new java.io.FileInputStream("arquivo.txt");
    }

    public static void main(String[] args) {
      metodo();
    }

}
```

Exceções Verificadas

- Não vai compilar! Tem que tratar! 2 opções:
  - Continua jogando pra frente

```
public static void main(String[] args) throws java.io.FileNotFoundException {
  metodo();
}
```

  - Usa try/catch

```
public static void main(String[] args){
  try {
      metodo();
  } catch (FileNotFoundException e) {
    e.printStackTrace();
  }
}
```

Exceções Verificadas

- No início, existe uma grande tentação de sempre passar o problema pra frente para outros o tratarem. Pode ser que faça sentido, dependendo do caso, mas não até o main, por exemplo.
- Acontece que quem tenta abrir um arquivo sabe como lidar com um problema na leitura.
- Quem chamou um método no começo do programa pode não saber ou, pior ainda, tentar abrir cinco arquivos diferentes e não saber qual deles teve um problema!

Exceções Verificadas

- Não há uma regra para decidir em que momento do seu programa você vai tratar determinada exceção. Isso vai depender de em que ponto você tem condições de tomar uma decisão em relação àquele erro.
- Enquanto não for o momento, você provavelmente vai preferir delegar a responsabilidade para o método que te invocou.

Lançando exceções

- Lembre-se do método saca da nossa classe Conta. Ele devolve um boolean caso consiga ou não sacar:

```
boolean saca(double valor) {
  if (this.saldo < valor) {
    return false;
  } else {
    this.saldo-=valor;
    return true;
  }
}
```

- Podemos, também, lançar uma *Exception*, o que é extremamente útil. Dessa maneira, resolvemos o problema de alguém esquecer de fazer um **if** no retorno de um método.

Lançando exceções

- A palavra chave throw, que está no imperativo, lança uma *Exception*. Isto é bem diferente de throws, que está no presente do indicativo, e que apenas avisa da possibilidade daquele método lançá-la, obrigando o outro método que vá utilizar deste de se preocupar com essa exceção em questão.

```
void saca(double valor) {
  if (this.saldo < valor) {
    throw new RuntimeException();
  } else {
    this.saldo-=valor;
  }
}
```

Lançando exceções

- Nesse caso, lança uma do tipo unchecked. *RuntimeException* é a exception mãe de todas as exceptions unchecked. A desvantagem, aqui, é que ela é muito genérica; quem receber esse erro não sabe dizer exatamente qual foi o problema. Podemos então usar uma Exception mais específica:

```
void saca(double valor) {
  if (this.saldo < valor) {
    throw new IllegalArgumentException();
  } else {
    this.saldo-=valor;
  }
}
```

Lançando exceções

- *IllegalArgumentException* diz um pouco mais: algo foi passado como argumento e seu método não gostou. Ela é uma Exception unchecked pois estende de RuntimeException e já faz parte da biblioteca do java.
- (*IllegalArgumentException* é a melhor escolha quando um argumento sempre é inválido como, por exemplo, números negativos, referências nulas, etc).

Lançando exceções

- Para pegar esse erro, não usaremos um if/else e sim um try/catch, porque faz mais sentido já que a falta de saldo é uma exceção:

```
Conta cc = new ContaCorrente();
cc.deposita(100);

try {
  cc.saca(100);
} catch (IllegalArgumentException e) {
  System.out.println("Saldo Insuficiente");
}
```

Lançando exceções

- Podemos melhorar ainda mais e passar para o construtor da *IllegalArgumentException* o motivo da exceção:

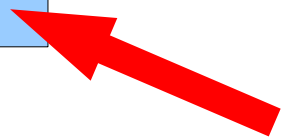
```
void saca(double valor) {
  if (this.saldo < valor) {
    throw new IllegalArgumentException("Saldo insuficiente");
  } else {
    this.saldo-=valor;
  }
}
```

Lançando exceções

- O método getMessage() definido na classe Throwable (mãe de todos os tipos de erros e *exceptions*) vai retornar a mensagem que passamos ao construtor da *IllegalArgumentException*.

método main

```
try {
  cc.saca(100);
} catch (IllegalArgumentException e) {
  System.out.println(e.getMessage());
}
```



O que colocar dentro do Try?

- Imagine que vamos sacar dinheiro de diversas contas:

```
Conta cc = new ContaCorrente();
cc.deposita(100);

Conta cp = new ContaPoupanca();
cp.deposita(100);

// sacando das contas:

cc.saca(50);
System.out.println("consegui sacar da corrente!");

cp.saca(50);
System.out.println("consegui sacar da poupança!");
```

O que colocar dentro do Try?

- Podemos escolher vários lugares para colocar try/catch:

```
try {
  cc.saca(50);
} catch (IllegalArgumentException e) {
  System.out.println(e.getMessage());
}
System.out.println("consegui sacar da corrente!");

try {
  cp.saca(50);
} catch (IllegalArgumentException e) {
  System.out.println(e.getMessage());
}
System.out.println("consegui sacar da poupança!");
```

O que colocar dentro do Try?

- Essa não parece uma opção boa, pois a mensagem "consegui sacar" será impressa mesmo que o catch seja acionado. Sempre que temos algo que depende da linha de cima para ser correto, devemos agrupá-lo no try:

```
try {
  cc.saca(50);
  System.out.println("consegui sacar da corrente!");
} catch (IllegalArgumentException e) {
  System.out.println(e.getMessage());
}

try {
  cp.saca(50);
  System.out.println("consegui sacar da poupança!");
} catch (IllegalArgumentException e) {
  System.out.println(e.getMessage());
}
```

O que colocar dentro do Try?

- Mas há ainda uma outra opção: imagine que, para o nosso sistema, uma falha ao sacar da conta poupança deve parar o processo de saques e nem tentar sacar da conta corrente. Para isso, agruparíamos mais ainda:

```
try {
  cc.saca(50);
  System.out.println("consegui sacar da corrente!");
  cp.saca(50);
  System.out.println("consegui sacar da poupança!");
} catch (IllegalArgumentException e) {
  System.out.println(e.getMessage());
}
```

O que colocar dentro do Try?

- Observação:
  - O que você vai colocar dentro do try influencia muito a execução do programa! Pense direito nas linhas que dependem uma da outra para a execução correta da sua lógica de negócios.

Exercícios

1. Utilize herança para criar uma superclasse de exceção (chamada **ExceptionA**) e subclasses de exceção **ExceptionB** e **ExceptionC**

- em que **ExceptionB** herda de **ExceptionA** e **ExceptionC** herda de **ExceptionB**.
- Escreva um programa para demonstrar que o bloco catch para tipo **ExceptionA** captura exceções de tipos **ExceptionB** e **ExceptionC**.